



Email Automation Engine

Complete guide — how the 24/7 cold-email + follow-up engine works, and how to keep it out of spam.

Brevo Email Pipeline · apollo-email-app · generated 21 Jul 2026

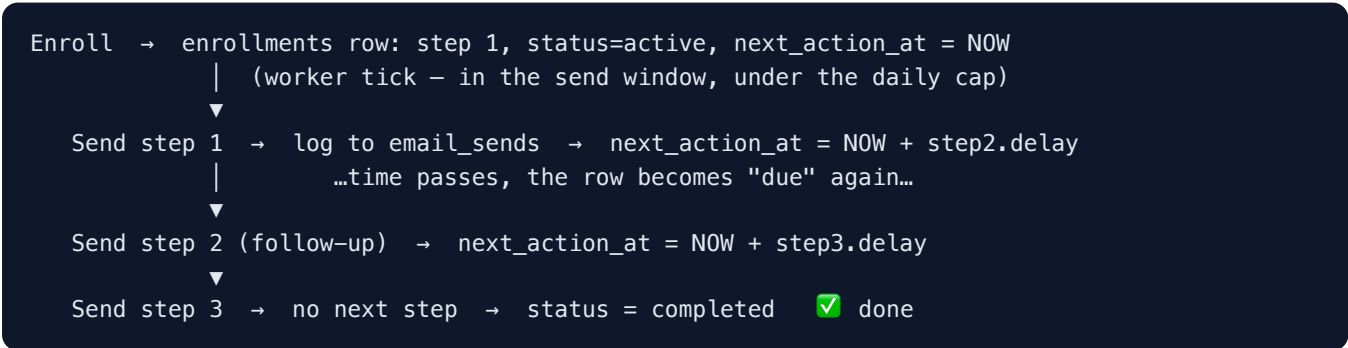
The one idea: a follow-up is *not* a separate system. Each contact in a campaign is one row in `enrollments` holding a `next_action_at` timestamp. When an email sends, we stamp `next_action_at = now() + next_step.delay`. A worker wakes every ~2 minutes and asks one question — *"who is active and due?"* — and sends them. The follow-up is just that row becoming due again. No AI decides timing; it is a SQL query and an `if`. AI is used in exactly one place: labeling inbound replies.

1 · The moving parts

The website and the automation are **two separate processes**. The site being up does **not** mean anything is sending — the worker is what sends.

Piece	What it is	Command
Postgres	Single source of truth: contacts, templates, sequences, campaigns, enrollments, suppressions, send log	—
Next app	UI + control-plane APIs + the public Brevo webhook + the unsubscribe page	<code>npm run dev</code>
Worker	The automation: send heartbeat + reply scan. Nothing sends unless this runs.	<code>npm run worker</code>
Brevo	The mail sender — one API call per email	external
IMAP + Claude	Optional reply detection: reads replies, classifies them	optional

2 · The lifecycle of one contact



At any point a stop condition ends the sequence early, re-checked before *every* send:

- Contact **replies** → status `replied`, added to suppressions.
- Hard **bounce** / spam complaint → status `bounced`, suppressed.
- **Unsubscribe** (link or reply) → status `unsubscribed`, suppressed.

- Already on the **suppression** list → skipped.

3 · What "automated" means — the heartbeat

The worker runs two `node-cron` timers: a **send tick** every `SEND_INTERVAL_MINUTES` (default 2) and a **reply scan** every `REPLY_INTERVAL_MINUTES` (default 5, if IMAP is set).

Each send tick, for every active campaign:

1. **Send window?** Current hour in the campaign's timezone must be within `window_start..window_end`.
2. **Under the daily cap?** Count today's sends (in that timezone); if `>= daily_cap`, skip.
3. **Claim due rows** in one transaction with `FOR UPDATE SKIP LOCKED` — so two overlapping ticks never grab the same contact (**no double-sends**).
4. **Send:** re-check stop conditions, render the step's template per contact (`{{first_name}}` ...), send via Brevo, **5 in flight** at a time.
5. **Log + advance** in the same transaction: write to `email_sends`, then set the next step / `next_action_at` (or `completed`). Committing together makes a mid-crash restart safe.

Reply scan & delivery events

Replies are read over IMAP, matched to an active enrollment, and classified by Claude into `interested` / `not_interested` / `out_of_office` / `unsubscribe` / `other` — then acted on in plain code (stop & suppress; out-of-office just defers a few days). Separately, Brevo *pushes* opens/clicks/bounces/unsubscribes to `/api/brevo-webhook`, which stamps the send row and suppresses on bounce/spam/unsub. **Follow-ups trigger on "no reply," never on "not opened"** — opens are unreliable.

4 · Running it — step by step

```
npm run db:setup      # 0. create the tables (idempotent)
# 1. fill in .env      (see .env.example – especially deliverability, §6)
npm run dev           # 2. start the app                (port 3001)
npm run worker        # 3. start the automation (separate process)
```

Then in the UI (**Campaigns** tab):

1. **Templates** (Recipients page) — write initial + follow-up emails with `{{first_name}}` etc.
2. **Sequence** — e.g. Step 1 (0h) → Step 2 (+48h) → Step 3 (+72h).
3. **Campaign** — pick the sequence, set an ILIKE filter, a **low** daily cap, and a send window. Created as a **draft** (nothing sends yet).
4. **Enroll** — previews the match count, then adds contacts.
5. **Start ▶** — flips to `active`; the running worker now sends.
6. Watch the **Sent** page + worker logs.

5 · The knobs (safety & throttling)

Setting	Where	Default	Purpose
daily_cap	per campaign	30	Max successful sends/day. Start low, ramp up.
window_start/end	per campaign	9–18	Only send during local business hours
timezone	per campaign	Asia/Kolkata	Whose clock the window/cap use
CONCURRENCY	worker (code)	5	Max Brevo requests in flight
SEND_INTERVAL_MINUTES	env	2	How often the send tick runs
000_DEFER_DAYS	env	3	How long out-of-office defers a contact

6 · Deliverability — the "it lands in spam" fix

This is **not a code bug** — it is **domain authentication**, and it must be done outside the app. The engine can send perfectly and still hit spam if the sending identity is not authenticated.

The root cause

Gmail/Outlook verify the sender is **authorized to send for the From domain** via **SPF, DKIM, and DMARC**. If they do not line up, the mail is spam-foldered or rejected, and Brevo often **rewrites the From** to something like `...@brevosend.com` — which looks untrustworthy and damages reputation further.

⚠ BLOCKING ISSUE IN YOUR CURRENT SETUP

`BREVO_SENDER_EMAIL` is currently a `@gmail.com` address. You **cannot** authenticate `gmail.com` (you don't own it), and Gmail publishes a strict `DMARC p=reject` policy for it. Sending "from" `@gmail.com` through Brevo therefore **fails DMARC by design** → Brevo rewrites the From → spam. Free webmail addresses (gmail / yahoo / outlook.com) can **never** be used as a cold-email From address. **This is the single biggest thing to fix.**

The fix — a domain you own (one-time, DNS + Brevo dashboard)

1. Use a **domain you own** (your company / radiusai domain), not gmail.com.
2. In Brevo → **Senders, Domains & Dedicated IPs**, add that domain and follow its instructions to add the **SPF, DKIM, and DMARC** DNS records at your registrar. Wait until Brevo shows the domain **authenticated** (green).
3. Set `BREVO_SENDER_EMAIL` to an address **on that domain** (e.g. `outreach@yourdomain.com`).
4. **Warm up**: keep `daily_cap` ~20–50/day for the first 2–3 weeks and grow slowly. Blasting thousands from a cold domain burns reputation.

✓ ALREADY HANDLED IN CODE

List-Unsubscribe + one-click List-Unsubscribe-Post headers on automated sends · a visible unsubscribe link + sender footer in every automated email · instant opt-out via the `suppressions` list, checked before every send · send windows + daily caps + 5-in-flight throttling.

🔵 STILL TO SWITCH ON (CONFIG, NOT CODE)

`APP_BASE_URL` is empty → the HTTP one-click unsubscribe link is not active yet (only the `mailto:` fallback). Set it to your public HTTPS URL once hosted, set `BREVO_WEBHOOK_SECRET`, and point Brevo's webhook at `/api/brevo-webhook?secret=...`. Optionally set `IMAP_*` (reply detection) and `ANTHROPIC_API_KEY` (reply classification).

7 · Automatic vs. manual — quick reference

Automatic worker running + campaign active

Sending the initial email and every follow-up on schedule · respecting window, cap & rate limit · stopping on reply / bounce / unsubscribe and suppressing them · stamping opens/clicks/ bounces from Brevo events · deferring out-of-office replies.

Manual / one-time

Authenticating the sending domain (the spam fix) · configuring the Brevo webhook URL + secret · setting `APP_BASE_URL` , `IMAP_*` , `ANTHROPIC_API_KEY` · creating templates / sequences / campaigns, enrolling, hitting **Start** · raising the daily cap as the domain warms.